Relatório Técnico Detalhado:
Sistema de Inventário e Drop Balanceado

Matheus Marques Eiras e Ciro

Abril de 2026

1 Introdução e Contexto do Projeto

O projeto consiste em um "Sistema de Inventário e Drop Balanceado"desenvolvido como parte dos requisitos da disciplina de Estruturas de Dados Avançadas do IFPR. O objetivo central do sistema é gerenciar eficientemente o armazenamento e a priorização de itens de jogos, integrando estruturas computacionais de alta performance com lógicas justas de geração procedural ("drops") baseadas em taxas dinâmicas de sorte e azar.

2 Referencial Teórico

A fundamentação deste sistema baseia-se em conceitos clássicos de computação e design de jogos:

- **Eficiência de Hashing:** Segundo Knuth [1], o Método da Multiplicação utilizando a Proporção Áurea ($\phi \approx 0,618$) é altamente eficaz para distribuir chaves uniformemente, minimizando colisões independentemente do tamanho da tabela.
- **Geradores Pseudo-Aleatórios (PRNGs):** O uso do algoritmo XORShift, proposto por Marsaglia [2], oferece uma alternativa superior aos LCGs tradicionais em termos de velocidade e qualidade estatística para aplicações de tempo real.
- **Mecânicas de Recompensa:** O sistema de "pity"implementado assemelha-se à Distribuição Pseudo-Aleatória (PRD), amplamente utilizada na indústria para mitigar a variância negativa e aumentar a satisfação do usuário [3].

3 Arquitetura das Estruturas de Dados

3.1 Unidade Básica de Dados (Item)

O dado fundamental que circula pelo sistema é o `Item` (definido no arquivo `item.hpp`), que atua como um *Data Transfer Object* (DTO) leve. Implementado como uma `struct`, possui os atributos `id`, `nome` e `raridade` (variando de 1 a 100). Um diferencial técnico é a sobrecarga dos operadores $<$ e $>$, permitindo que o `Max-Heap` organize as instâncias nativamente pela raridade.

3.2 Armazenamento e Busca (Tabela Hash)

A `HashTable` atua como o inventário principal, garantindo uma busca de $O(1)$ no caso médio. Possui tamanho base primo de 101 e utiliza encadeamento externo via `std::list`. O sistema oferece dois métodos de espalhamento:

- **Método da Divisão:** $h(k) = k \mod m$.
- **Método da Multiplicação:** Baseado na Proporção Áurea ($A \approx 0,618$), utilizando a fórmula:
$$h(k) = \lfloor m(k \cdot A \mod 1) \rfloor$$

3.3 Priorização e Extração (Max-Heap)

Para acessar o item de maior raridade em tempo constante $O(1)$ ou retirá-lo em $O(\log n)$, utilizou-se um `Max-Heap` suportado por um `std::vector`. A matemática de índices define os filhos de i como $2i + 1$ e $2i + 2$, permitindo rebalanceamento eficiente via *siftUp* e *siftDown*.

4 Geração de Aleatoriedade e PRNGs

O motor probabilístico baseia-se em Geradores de Números Pseudo-Aleatórios controlados via classe base abstrata `PRNG`, garantindo reprodutibilidade via *seeds*:

- **LCG (Linear Congruential Generator):** Utiliza as constantes $a = 1664525$ e $c = 1013904223$.
- **XORShift:** Operado via *bitwise*. Possui uma rede de segurança que força a semente para 42 caso o valor 0 seja informado.

5 Mecanismo Adaptativo de Drop Balanceado

A classe `DropSystem` mantém a distribuição de itens raros fixada em 10% (*targetRatio* = 0,1). Sempre que um item supera o *rareThreshold* (80), o sistema recalcula o *ratio* efetivo:

- **Sorte alta:** `currentRareChance` é multiplicado por 0,95.
- **Azar extremo:** O limite de probabilidade é acrescido até atingir o *cap* de 50% (sistema de *pity*).

6 Orquestração e Validação de Fluxo

O arquivo `main.cpp` provê captura segura de argumentos CLI (ex: `-seed 123`). A simulação padrão gera 20 itens processados simultaneamente, extraindo os 3 itens de maior raridade e validando o rastreio do item ID 10 na `HashTable`.

7 Benchmarks e Análise de Desempenho

Os testes de estresse foram conduzidos em três escalas de volume de dados (1k, 10k e 100k). Observou-se uma consistência notável na eficácia do algoritmo de drop, mantendo a proporção de itens raros próxima ao alvo de 10%.

Métrica	**1.000 itens**	**10.000 itens**	**100.000 itens**
Tempo Total de Processamento	0,114 ms	1,435 ms	35,009 ms
Colisões (Divisão / Multipl.)	500 / 500	5.000 / 5.000	50.000 / 50.000
Proporção Real de Itens Raros	10,00%	9,98%	9,99%
Tempo (1k buscas na Hash)	0,001175 ms	0,001175 ms	0,001176 ms
Extração (1k itens do Heap)	0,048822 ms	0,093714 ms	0,419225 ms

Tabela 1: Resultados comparativos de performance por volume de carga.

8 Especificações do Ambiente de Teste

Para garantir a reprodutibilidade dos resultados de benchmark, detalham-se as especificações utilizadas:

- **Processador:** AMD Ryzen 5 9600X 6-Core (Frequência Base: 3.90 GHz).
- **Memória RAM:** 32,0 GB.
- **Sistema Operacional:** Windows 11 Pro, Versão 25H2.

9 Conclusão

A arquitetura consolida um microssistema robusto para backends de itens digitais. As decisões de design resultaram em uma modelagem performática e justa, apresentando tempos de busca quase constantes e uma distribuição de raridade estatisticamente saudável.

Referências

[1] KNUTH, Donald E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Reading, MA: Addison-Wesley, 1998.

[2] MARSAGLIA, George. *Xorshift RNGs*. Journal of Statistical Software, Vol. 8, Issue 14, 2003.

[3] GAREA, S. S. et al. *Loot boxes – gambling-like mechanisms hidden in digital games*. Rozprawy Społeczne, 2026.